Spring Design Presentation: Tenamore Wedding Planning Application

TENAMORE Estates

A Full-Stack Web Application designed to help wedding planners manage their clients more efficiently

Names:

Team Members:

Michael Weinstein

Weinstmd@mail.uc.edu

Advisor:

Dr. Will Hawkins

Hawkinwh@ucmail.uc.edu

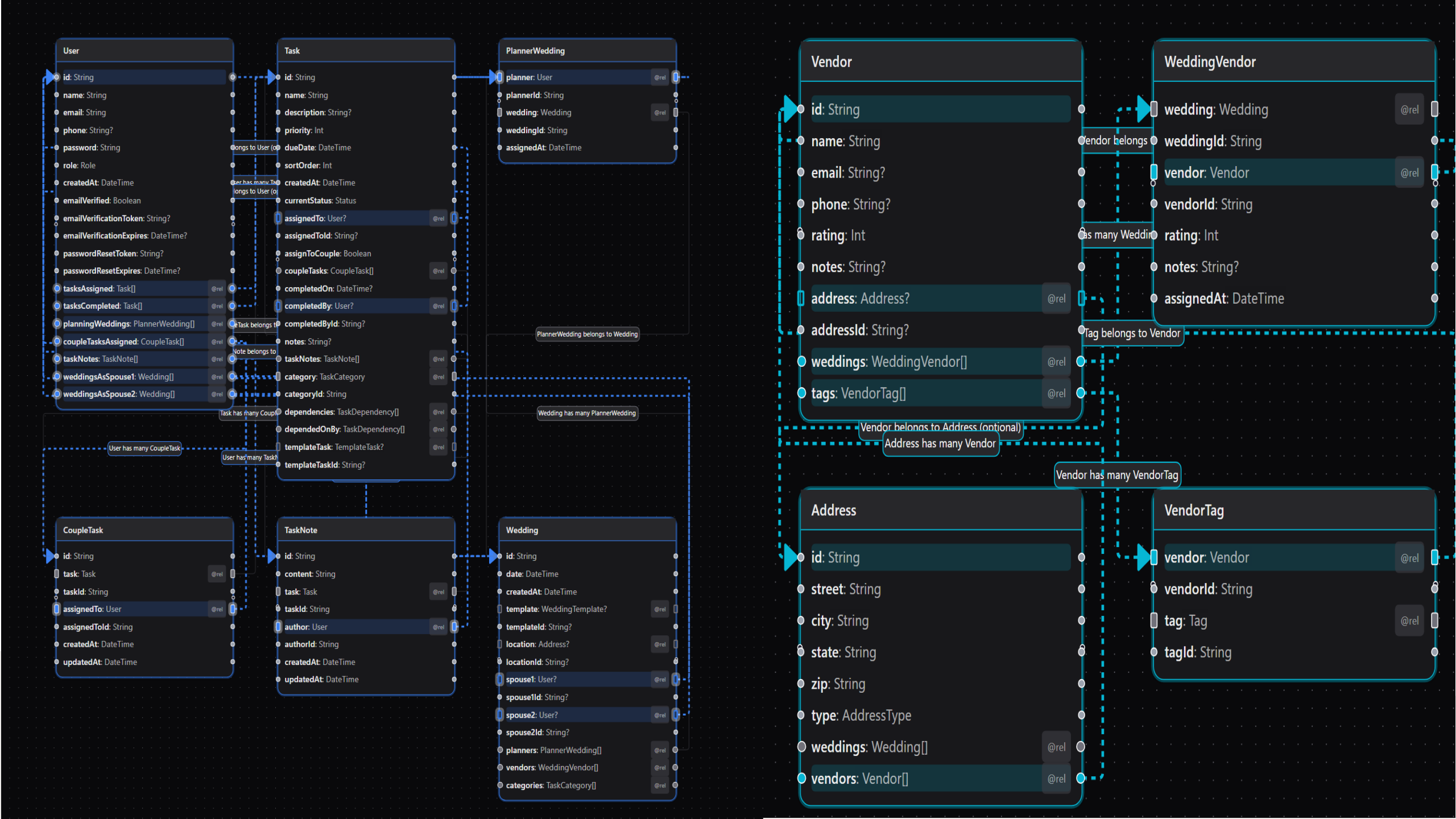
Goals

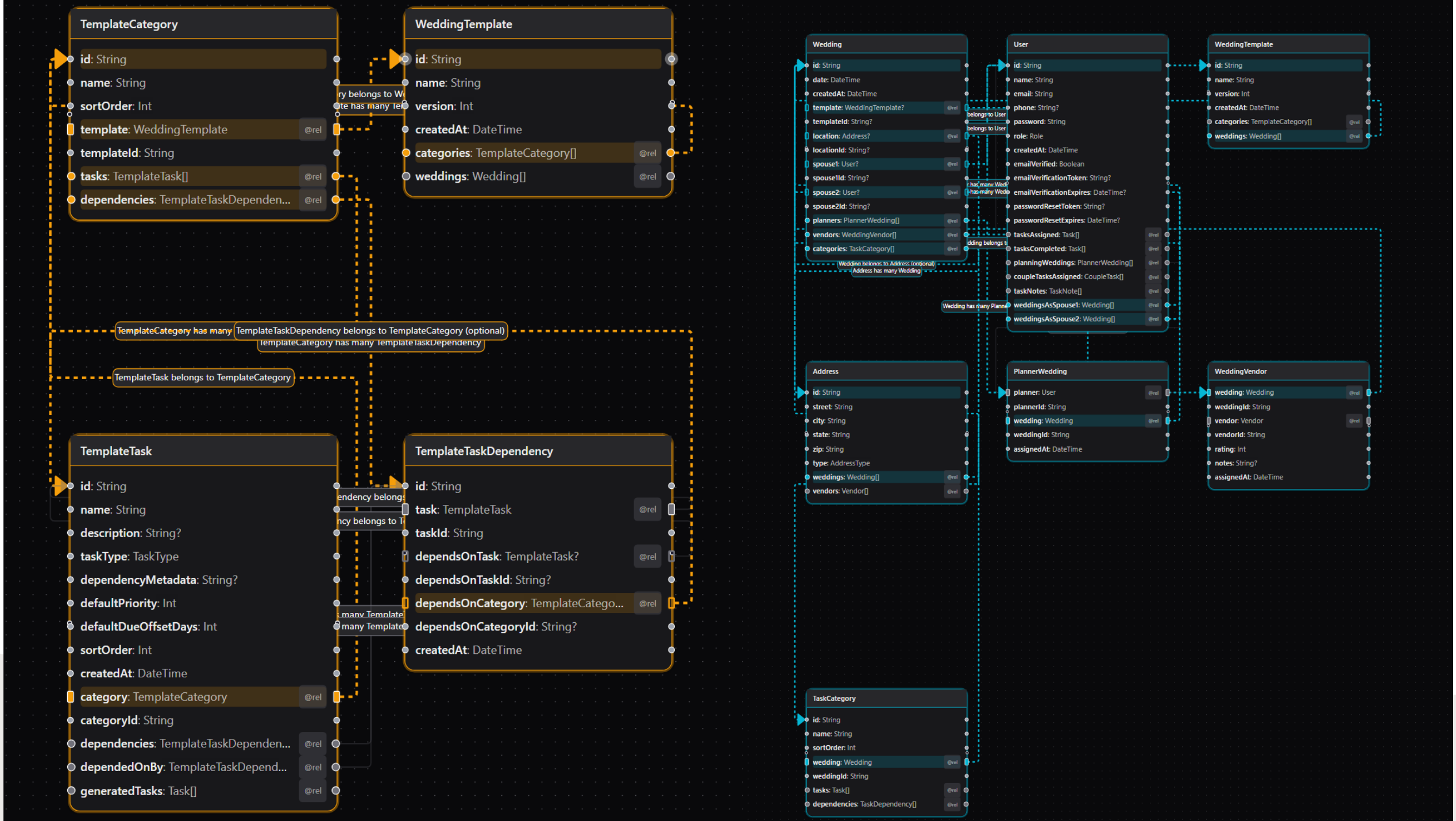
Wedding planning often involves fragmented communication and manual scheduling across multiple parties. Tenamore seeks to solve this problem by providing a centralized platform that automates scheduling, streamlines vendor and guest communication, and organizes key planning milestones. Through a responsive React frontend, a Node.js backend, and an integrated database, the system enables real-time updates, task management, and automatic conflict detection.

Why Build a Wedding Planning Application?

My sister has a wedding planning business, and is opening up a venue in Nashville, TN.

The job market is truly awful right now, and building a project that will have real-world application I can point to feels like a way to set myself apart – I'm not a fantastic programmer, but I can build working apps!


User
id: String
name: String
email: String
phone: String?
password: String
role: Role
createdAt: DateTime
emailVerified: Boolean
emailVerificationToken: String?
emailVerificationExpires: DateTime?
passwordResetToken: String?
passwordResetExpires: DateTime?
tasksAssigned: Task[]
tasksCompleted: Task[]
planningWeddings: PlannerWedding[]
coupleTasksAssigned: CoupleTask[]
taskNotes: TaskNote[]
weddingsAsSpouse1: Wedding[]
weddingsAsSpouse2: Wedding[]
Task
id: String
name: String
description: String?
priority: Int
dueDate: DateTime
sortOrder: Int
createdAt: DateTime
currentStatus: Status
assignedTo: User?
assignedToId: String?
assignToCouple: Boolean
coupleTasks: CoupleTask[]
completedOn: DateTime?
completedBy: User?
completedById: String?
notes: String?
taskNotes: TaskNote[]
category: TaskCategory
categoryId: String
dependencies: TaskDependency[]
dependedOnBy: TaskDependency[]
templateTask: TemplateTask?
templateTaskId: String?
PlannerWedding
planner: User
plannerId: String
wedding: Wedding
weddingId: String
assignedAt: DateTime
PlannerWedding belongs to Wedding
Wedding has many PlannerWedding
User has many CoupleTask
CoupleTask
id: String
task: Task
taskId: String
assignedTo: User
assignedToId: String
createdAt: DateTime
updatedAt: DateTime
TaskNote
id: String
content: String
task: Task
taskId: String
author: User
authorId: String
createdAt: DateTime
updatedAt: DateTime
Wedding
id: String
date: DateTime
createdAt: DateTime
template: WeddingTemplate?
templateId: String?
location: Address?
locationId: String?
spouse1: User?
spouse1Id: String?
spouse2: User?
spouse2Id: String?
planners: PlannerWedding[]
vendors: WeddingVendor[]
categories: TaskCategory[]
Vendor
id: String
name: String
email: String?
phone: String?
rating: Int
notes: String?
address: Address?
addressId: String?
weddings: WeddingVendor[]
tags: VendorTag[]
WeddingVendor
wedding: Wedding
weddingId: String
vendor: Vendor
vendorId: String
rating: Int
notes: String?
assignedAt: DateTime
Tag belongs to Vendor
Vendor belongs to Address (optional)
Address has many Vendor
Vendor has many VendorTag
Address
id: String
street: String
city: String
state: String
zip: String
type: AddressType
weddings: Wedding[]
vendors: Vendor[]
VendorTag
vendor: Vendor
vendorId: String
tag: Tag
tagId: String


TemplateCategory
id: String
name: String
sortOrder: Int
template: WeddingTemplate @rel
templateId: String
tasks: TemplateTask[] @rel
dependencies: TemplateTaskDependen... @rel
WeddingTemplate
id: String
name: String
version: Int
createdAt: DateTime
categories: TemplateCategory[] @rel
weddings: Wedding[] @rel
TemplateCategory has many
TemplateTaskDependency belongs to TemplateCategory (optional)
TemplateCategory has many TemplateTaskDependency
TemplateTask belongs to TemplateCategory
TemplateTask
id: String
name: String
description: String?
taskType: TaskType
dependencyMetadata: String?
defaultPriority: Int
defaultDueOffsetDays: Int
sortOrder: Int
createdAt: DateTime
category: TemplateCategory @rel
categoryId: String
dependencies: TemplateTaskDependen... @rel
dependedOnBy: TemplateTaskDepend... @rel
generatedTasks: Task[] @rel
TemplateTaskDependency
id: String
task: TemplateTask @rel
taskId: String
dependsOnTask: TemplateTask? @rel
dependsOnTaskId: String?
dependsOnCategory: TemplateCatego... @rel
dependsOnCategoryId: String?
createdAt: DateTime
Wedding
id: String
date: DateTime
createdAt: DateTime
template: WeddingTemplate? @rel
templateId: String?
location: Address? @rel
locationId: String?
spouse1: User? @rel
spouse1Id: String?
spouse2: User? @rel
spouse2Id: String?
planners: PlannerWedding[] @rel
vendors: WeddingVendor[] @rel
categories: TaskCategory[] @rel
belongs to User
belongs to User
Wedding belongs to Address (optional)
Address has many Wedding
Wedding has many Planne
User
id: String
name: String
email: String
phone: String?
password: String
role: Role
createdAt: DateTime
emailVerified: Boolean
emailVerificationToken: String?
emailVerificationExpires: DateTime?
passwordResetToken: String?
passwordResetExpires: DateTime?
tasksAssigned: Task[] @rel
tasksCompleted: Task[] @rel
planningWeddings: PlannerWedding[] @rel
coupleTasksAssigned: CoupleTask[] @rel
taskNotes: TaskNote[] @rel
weddingsAsSpouse1: Wedding[] @rel
weddingsAsSpouse2: Wedding[] @rel
WeddingTemplate
id: String
name: String
version: Int
createdAt: DateTime
categories: TemplateCategory[] @rel
weddings: Wedding[] @rel
Address
id: String
street: String
city: String
state: String
zip: String
type: AddressType
weddings: Wedding[] @rel
vendors: Vendor[] @rel
PlannerWedding
planner: User @rel
plannerId: String
wedding: Wedding @rel
weddingId: String
assignedAt: DateTime
WeddingVendor
wedding: Wedding @rel
weddingId: String
vendor: Vendor @rel
vendorId: String
rating: Int
notes: String?
assignedAt: DateTime
TaskCategory
id: String
name: String
sortOrder: Int
wedding: Wedding @rel
weddingId: String
tasks: Task[] @rel
dependencies: TaskDependency[] @rel

Technologies

Tech Stack:

- PostgreSQL database
- Node.js backend
- Externally connected APIs for Sendgrid
- HTML/CSS/JS for frontend, using React to make development easier

Hosting Services:

- Database hosted on Supabase
- Backend served by Render
- Frontend deployed using Vercel

Expected Demo at Expo

I look forward to demonstrating a fully functioning application at UC's Design Expo. This will include:

- A bespoke frontend ui with admin, planner, and client dashboards
- Template logic for automated wedding planning from wedding date
- The application is hosted on Supabase, Vercel, and Render, providing a web-based interface accessible from anywhere in the world